

DLLM — A Distributed, Modular, Scalable and Efficient LLMs Design

Bin Xie

Independent Researcher
yorkxie@outlook.com

Kai De Xie

University of Waterloo
kdxie@uwaterloo.ca

Abstract

This paper introduces DLLM, a new way to design LLM systems that is distributed, modular, scalable, and efficient. DLLM separates language understanding, reasoning, and expression into independent modules. These modules communicate in human-readable language using a syntax defined by a shared Basic Language (BL) — a small language built from about 1000 well-selected and clearly defined keywords. All human languages are translated into BL in the pre-processing module. BL is used to communicate within the internal distributed LLM modules. Post-processing LLMs translate BL back into human language. This makes the internal LLM simpler and more efficient. DLLM supports distributed training, distributed inference, plug-and-play LLM modules, LLM distributed networking, and LLM module replacement.

1 Introduction

Modern LLMs are trained end-to-end on huge datasets. They must learn grammar, vocabulary, domain jargon, world knowledge, reasoning, style, formatting, and multiple languages inside one giant model. This “everything in one place” design leads to extremely large models, high training cost, difficulty updating or specializing, limited modularity, poor interpretability, and tight coupling between unrelated tasks.

DLLM takes a different approach. Instead of one giant model, DLLM is a pipeline of smaller, specialized models, each with a clear role. All modules communicate using Basic Language (BL), a simple, stable, universal meaning-language.

2 Motivation

Today’s LLMs spend a huge amount of capacity on tasks that are not reasoning: memorizing vocabulary, handling typos and slang, learning grammar rules, mapping synonyms, and dealing with domain-specific terms. DLLM removes this burden by introducing BL — a small, universal meaning-language built from about 1000 well-selected and clearly defined keywords.

Once everything is translated into BL, the reasoning model no longer needs to understand grammar, vocabulary, or domain jargon. It only needs to understand meaning. This separation leads to smaller reasoning models, easier training, modular upgrades, distributed execution across processors, domain isolation, and long-term maintainability.

3 DLLM Architecture Overview

DLLM has five main components.

3.1 Pre-Processing LLMs

The pre-processing module converts raw human language into BL. It handles multilingual input, slang, abbreviations, and grammar normalization. It also inserts a BL keyword indicating the source language (e.g., `LANG=English`). Domain-specific jargon is passed through as tagged strings, which are preserved exactly and only processed by the appropriate professional LLM.

3.2 Basic Language (BL)

BL is a small, stable meaning-language built from about 1000 well-selected and clearly defined keywords. It represents semantic roles, actions, entities, relationships, logical structure, intent, and modality. BL removes grammar, synonyms, morphology, domain jargon, and language differences.

3.3 Reasoning LLM (Core Engine)

The reasoning LLM is the central controller of DLLM. It operates entirely in BL and performs all high-level reasoning, planning, and decision-making. It does not interpret tagged strings. It can call professional LLMs, loop outputs back into them, and decide when reasoning is complete.

3.4 Professional LLMs (Domain Modules)

Professional LLMs specialize in fields such as medical, legal, programming, finance, engineering, and scientific domains. They operate under the control of the reasoning LLM. They receive BL input, tagged strings, and optional language tags. They may produce tagged strings in their output. The reasoning LLM may loop outputs through multiple professional LLMs as needed.

3.5 Post-Processing LLMs

The post-processing module converts BL back into natural language. It handles grammar, style, tone, domain phrasing, and formatting. It does not perform deep reasoning.

4 Distributed Training

DLLM naturally supports distributed training because each module is independent. Pre-processing LLMs, domain LLMs, the reasoning core, and post-processing LLMs can all be trained separately. BL is the shared contract.

5 Distributed Inference

DLLM supports distributed inference: pre-processing LLMs can run on different machines, BL can be routed across nodes, the reasoning core can run on a small GPU, and professional LLMs can run on separate devices. BL is tiny, so routing is cheap.

6 BL Design Principles

BL must be small, stable, unambiguous, language-neutral, domain-neutral, and meaning-focused. It is inspired by semantic roles, predicate logic, and conceptual graphs, but simplified for LLMs.

7 Comparison to Existing LLM Architectures

DLLM differs from monolithic LLMs in language understanding, reasoning, grammar handling, vocabulary size, training method, scaling strategy, upgradeability, and interoperability. DLLM is not MoE, not RAG, and not a transformer variant.

8 Advantages

DLLM provides modularity, scalability, efficiency, maintainability, interoperability, multilingual unification, domain isolation, distributed execution across processors, and loop-capable reasoning.

9 Limitations and Challenges

DLLM introduces challenges such as designing BL, training high-quality pre-processing translators, ensuring BL coverage, and building a router to choose the right professional LLM. These are engineering challenges, not theoretical barriers.

10 Future Work

Future directions include formalizing BL, building open BL standards, training the first DLLM prototype, comparing DLLM to monolithic LLMs, exploring cross-organization module sharing, and designing BL-native reasoning algorithms.

11 Conclusion

DLLM is a new way to build LLM systems: distributed, modular, scalable, and efficient. By separating language understanding, reasoning, and expression — and by introducing a universal Basic Language (BL) — DLLM reduces complexity and opens the door to a new generation of AI systems that are easier to train, maintain, and scale.

DLLM is not a small improvement. It is a structural redesign of how LLMs should be built.